

Database Design Document

Project: MoMo SMS Data Parser

Team: Parsers

Members: James Dovee Kanneh II, Gavin GANZA, Herve IMENA Rwigema, Yai Majak D'Agoot

Database name: momo_sms_db

Schema source: database/database_setup.sql

1. Entity-Relationship Diagram (ERD) Documentation

The schema has **five** tables. *transactions* is the central entity.

users: Stores people and businesses that appear in MoMo SMS: customers, agents, merchants.

transactions: One row per recorded financial event. This table holds amount, fee, balance after (when the SMS states it), status, transaction date, optional MoMo reference id, and the full raw sms text.

transaction_categories: Lookup for operation type: deposit, payment, transfer, bank deposit, airtime, withdrawal, and similar labels the ETL assigns after reading the SMS.

transaction_participants: This is the junction between *users* and *transactions*. Each row says which user played which role on that transaction (sender, receiver, account owner, agent, merchant, or other).

system_logs: This table holds the audit trails. Most rows point at a *transaction_id*, but the column is nullable so we can log warnings that are not tied to a single transaction yet.

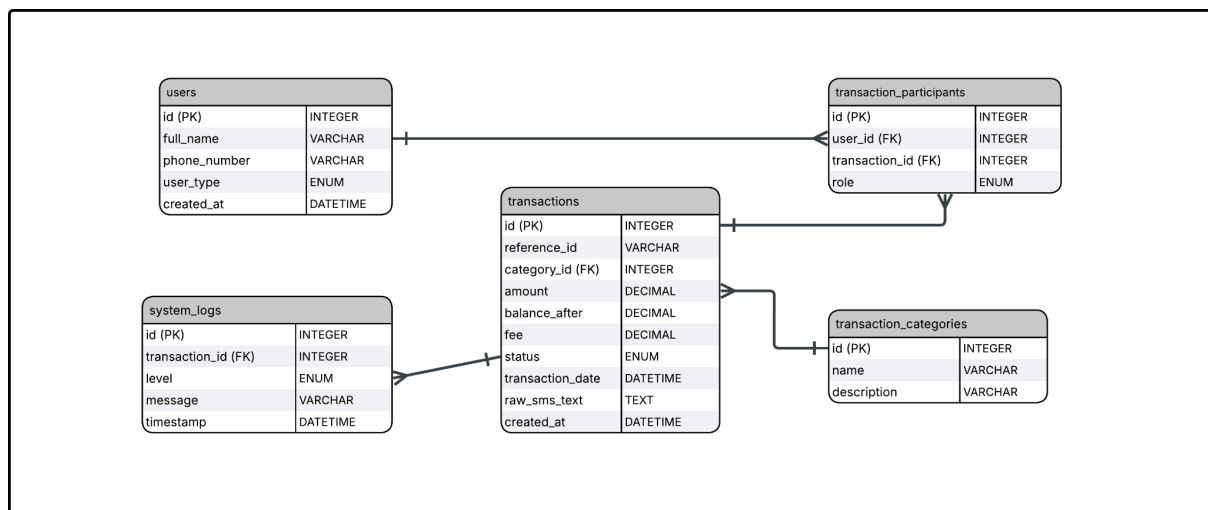
Users and transactions are many-to-many. We resolve that through ***transaction_participants***.

Relationships

Each transaction belongs to exactly one category. Each category can appear on many transactions.

Each transaction can have many log entries. Each log entry references at most one transaction.

Entity relationship diagram



2. Design rationale and justification

Mobile-money SMS describe money movements between people, agents, and merchants, but the wording and fields differ by message type. A single SMS can describe more than two parties involved. For example, a bank deposit credited to an account owner but facilitated by an agent, or an airtime purchase paid to a merchant.

A few decisions in our design include:

- Transaction types live in **transaction_categories** instead of being copied on every row.
- User identities live in **users** once; involvement in each event is recorded in **transaction_participants** with a role ENUM. That avoids duplicating sender and receiver columns on transactions.

- Each SMS is one **transaction** row. We keep raw_sms_text on that row because the SMS is the source of truth.

To ensure data integrity, most data rules live in the SQL schema, not only in Python code. Examples for these rules include:

- Categories cannot be deleted while transactions still reference them.
- Deleting a transaction deletes its participant rows.
- A unique key on (user_id, transaction_id, role) stops duplicate participant rows from a bad double insert

3. Data dictionary

users

People or entities identified from SMS content.

- **id**: INT, primary key, auto increment. Internal id.
- **full_name**: VARCHAR(200), required. Name as extracted from the SMS.
- **phone_number**: VARCHAR(20), optional. Normalized number when present.
- **user_type**: ENUM: CUSTOMER, MERCHANT, AGENT, UNKNOWN. Default UNKNOWN.
- **created_at**: DATETIME. Default current timestamp when the row is inserted.

transaction_categories

Classifies each transaction for reporting and the dashboard.

- **id**: INT, primary key, auto increment.
- **name**: VARCHAR(100), required. Short code such as DEPOSIT or TRANSFER.
- **description**: VARCHAR(255), optional. Longer label for humans.

transactions

Core fact table. One row per parsed SMS event.

- **id**: INT, primary key, auto increment.
- **reference_id**: VARCHAR(100), optional. MoMo TxId when the SMS provides one.
- **category_id**: INT, required. Foreign key to `transaction_categories.id`, ON DELETE RESTRICT.
- **amount**: DECIMAL(15,2), required. Must be greater than zero (CHECK).
- **balance_after**: DECIMAL(15,2), optional. Parsed from SMS when available.
- **fee**: DECIMAL(15,2), required, default 0. Must be zero or positive (CHECK).
- **status**: ENUM: COMPLETED, FAILED, PENDING, REVERSED, UNKNOWN. Default UNKNOWN.
- **transaction_date**: DATETIME, required. Event time from the SMS, not insert time.
- **raw_sms_text**: TEXT, required. Full original message.
- **created_at**: DATETIME. Default current timestamp when loaded.

Index: **idx_transactions_category** on **category_id**.

transaction_participants

Links users to transactions with a role.

- **id**: INT, primary key, auto increment.
- **user_id**: INT, required. Foreign key to `users.id`, ON DELETE CASCADE.
- **transaction_id**: INT, required. Foreign key to `transactions.id`, ON DELETE CASCADE.
- **role**: ENUM: SENDER, RECEIVER, ACCOUNT_OWNER, MERCHANT, AGENT, OTHER. Required.

Unique constraint **uq_participant** on (**user_id**, **transaction_id**, **role**).

Indexes: **idx_participants_user** on **user_id**; **idx_participants_transaction** on **transaction_id**.

system_logs

Parser and ETL audit messages.

- **id**: INT, primary key, auto increment.
- **transaction_id**: INT, optional. Foreign key to **transactions.id**, ON DELETE SET NULL.
- **level**: ENUM: INFO, WARNING, ERROR. Required.
- **message**: VARCHAR(800), required. What happened or what failed.
- **timestamp**: DATETIME. Default current timestamp.

Index: **idx_logs_transaction** on **transaction_id**.

4. Sample queries demonstrating database functionality

We run the queries against the schema and sample data defined in **database/**. Before running the demos, load them with:

```
mysql -u root -p < database/database_setup.sql  
mysql -u root -p momo_sms_db < database/insert_test_data.sql  
mysql -u root -p momo_sms_db < database/crud_tests.sql
```

Full terminal output from **crud_tests.sql** is saved as **docs/crud_output.txt** in the project repository.

Sample Queries

1. All transactions with category names.
2. Total amount per category.
3. All parties on transaction id **6**.

4. Transactions where **Jean Pierre Uwimana** is **SENDER**
5. Insert new withdrawal between a customer and an agent.
6. Update transaction status to REVERSED.
7. Update **Marie Claire Mukamana** phone number.
8. Delete test transaction; participant count goes from 2 to 0

Query results

All transactions with category name						
	reference_id	category	amount	status	transaction_date	
1	76662021700	DEPOSIT	2000.00	COMPLETED	2024-05-10 16:30:51	
2	73214846437	PAYMENT	1000.00	COMPLETED	2024-05-10 16:31:39	
3	51732411227	PAYMENT	600.00	COMPLETED	2024-05-10 21:32:32	
4	NULL	BANK_DEPOSIT	40000.00	COMPLETED	2024-05-11 18:43:49	
5	17818959211	PAYMENT	2000.00	COMPLETED	2024-05-11 18:48:42	
6	NULL	TRANSFER	10000.00	COMPLETED	2024-05-11 20:34:47	
7	NULL	TRANSFER	1000.00	COMPLETED	2024-05-12 03:47:33	
8	13913173274	AIRTIME	2000.00	COMPLETED	2024-05-12 11:41:28	

Total amount moved per category		
category	txn_count	total_amount
BANK_DEPOSIT	1	40000.00
TRANSFER	2	11000.00
PAYMENT	3	3600.00
AIRTIME	1	2000.00
DEPOSIT	1	2000.00

All parties involved in transaction 6 (transfer)

full_name	user_type	role
Jean Pierre Uwimana	CUSTOMER	SENDER
Eric Nkurunziza	CUSTOMER	RECEIVER

Every transaction Jean Pierre Uwimana sent

id	reference_id	amount	transaction_date
2	73214846437	1000.00	2024-05-10 16:31:39
3	51732411227	600.00	2024-05-10 21:32:32
5	17818959211	2000.00	2024-05-11 18:48:42
6	NULL	10000.00	2024-05-11 20:34:47
7	NULL	1000.00	2024-05-12 03:47:33
8	13913173274	2000.00	2024-05-12 11:41:28

CRUD results (Q5-Q8)

```
INSERT a new withdrawal transaction
reference_id category_id amount balance after fee status transaction_date
row_name_text created_at
9 999999999999 6 5000.00 20280.00 100.00 COMPLETED 2024-05-13 09:15:00
You have withdrawn 5000 RWF from MoMo Agent Remera 36521838. Fee was 100 RWF. New balance: 20280 RWF.
2026-09-15 21:31:12

UPDATE a transaction status (flagged as REVERSED)
id status
9 REVERSED

UPDATE a user phone number
id full_name phone_number
2 Marie Claire Mukamana 250788999888

DELETE a transaction (CASCADE into transaction.participants)
participants_before
2
participants_after
0
```

5. Rules for security, accuracy, and integrity

These constraints are defined in **database_setup.sql** and tested by deliberately adding inserts and deletes. The screenshots below show MySQL rejecting the invalid operations.

Negative amount insert blocked

```
jamesdkanneh8@penguin:~/momo_sms$ sudo mysql -u root momo_sms_db -e "INSERT INTO transactions (category_id, amount, transaction_date, raw_sms_text) VALUES (1, -50.00, NOW(), 'test');"
-----
INSERT INTO transactions (category_id, amount, transaction_date, raw_sms_text) VALUES (1, -50.00, NOW(), 'test')
-----

ERROR 4025 (23000) at line 1: CONSTRAINT `chk_transactions_amount` failed for `momo_sms_db`.`transactions`
jamesdkanneh8@penguin:~/momo_sms$
```

Duplicate participant insert blocked

```
jamesdkanneh8@penguin:~/momo_sms$ sudo mysql -u root momo_sms_db -e "INSERT INTO transactions (category_id, amount, transaction_date, raw_sms_text) VALUES (999, 500.00, NOW(), 'test');"
-----
INSERT INTO transactions (category_id, amount, transaction_date, raw_sms_text) VALUES (999, 500.00, NOW(), 'test')
-----

ERROR 1452 (23000) at line 1: Cannot add or update a child row: a foreign key constraint fails (`momo_sms_db`.`transactions`, CONSTRAINT `transactions_ibfk_1` FOREIGN KEY (`category_id`) REFERENCES `transaction_categories` (`id`))
jamesdkanneh8@penguin:~/momo_sms$
```

Invalid category foreign key blocked

```
jamesdkanneh8@penguin:~/momo_sms$ sudo mysql -u root momo_sms_db -e "INSERT INTO transactions (category_id, amount, transaction_date, raw_sms_text) VALUES (1, -50.00, NOW(), 'test');"
-----
INSERT INTO transactions (category_id, amount, transaction_date, raw_sms_text) VALUES (1, -50.00, NOW(), 'test')
-----

ERROR 4025 (23000) at line 1: CONSTRAINT `chk_transactions_amount` failed for `momo_sms_db`.`transactions`
jamesdkanneh8@penguin:~/momo_sms$
```

Category delete blocked while in use

```
jamesdkanneh8@penguin:~/momo_sms$ sudo mysql -u root momo_sms_db -e "INSERT INTO transactions (category_id, amount, transaction_date, raw_sms_text) VALUES (999, 500.00, NOW(), 'test');"
-----
INSERT INTO transactions (category_id, amount, transaction_date, raw_sms_text) VALUES (999, 500.00, NOW(), 'test')
-----

ERROR 1452 (23000) at line 1: Cannot add or update a child row: a foreign key constraint fails (`momo_sms_db`.`transactions`, CONSTRAINT `transactions_ibfk_1` FOREIGN KEY (`category_id`) REFERENCES `transaction_categories` (`id`))
jamesdkanneh8@penguin:~/momo_sms$
```